Experiment No: 07

Student Name: Asmit Prabhakar
UID: 23BCS10063
Branch: BE-CSE
Section/Group: KRG_2B
Semester: 6th
Date of Performance: 25/03/2026
Subject Code: 23CSH-314
Subject Name: System Design

Aim:

To design and analyze a scalable ride-sharing application that enables users to book rides and drivers to accept and fulfill requests in real time, ensuring high availability, low latency, and reliability at large scale.

Objective:

- To understand system design principles for large-scale ride-sharing systems.
- To design APIs for ride booking, driver matching, and payment services.
- To identify functional and non-functional requirements.
- To apply distributed system concepts such as CAP theorem.
- To design a high-availability and low-latency system architecture.

Procedure:

- Identify functional requirements of a ride-sharing application.
- Define non-functional requirements such as scalability, latency, and availability.
- Analyze CAP theorem trade-offs for real-time ride matching.
- Identify core entities such as Rider, Driver, Ride, and Payment.
- Design the system architecture using Draw.io.
- Validate the design for real-time ride booking and tracking scenarios.

Functional Requirements:

User Management
- User should be able to register.
- User should be able to login/logout.
- User should be able to update profile information.

Ride Booking
- User should be able to enter pickup and drop location.
- System should provide fare estimation.
- User should be able to request a ride.
- User should be able to choose ride type (car, bike, etc.).

Driver Management
- Driver should be able to register and login.
- Driver should be able to accept/reject ride requests.
- Driver should be able to update availability status.

Ride Management

- System should match riders with nearby drivers.
- System should provide real-time ride tracking.
- System should maintain ride history.
- System should allow cancellation of rides.

Payment & Rating

- System should support online/offline payments.
- System should generate invoices after ride completion.
- User should be able to rate drivers.
- Driver should be able to rate users.

Core Entities of the System:

- Users/ Riders
- Driver
- Location (Pick-Up and Drop-Off)
- Fare
- Ride/ Trip

API Design:

User / Rider

- [Get Request to check the fare]
- GET: /api/fare? pickupLatitude=? & pickupLongitude=? & dropLatitude=? & dropLongitude=?
- Response: List. <Fare> with requestID
- [User will be sending the requesID in backend -> which triggers notification to all drivers present nearby]
- POST: /api/ride / request

  req body:
  {
  requesID
  }
  res body:
  {
  ridelD,
  driverID (if driver accepts the ride)
  other meta data of driver
  }
- GET:/api/rides / history
- POST: /api/rides / {ridelD} / cancel
- POST: /api/rides / {rideID) / ratings

Driver

Driver's Location

- WS: / drivers / location / {body: Lat, Long} - update Driver's geoLocation [Persistent Connection is required]
- POST: /api/ride / rides / {body: requestID, accept / deny} - driver will accept / deny - return as ridelD
- POST: /api/ride / {ride_id} / start {
  "timestamp": "2026-03-22T17:30:00Z",

"start_lat": 30.7333,
"start_long": 76.7794,
"odometer_reading": 15240.5
}

- POST: /api/ride / {ride_id} / end

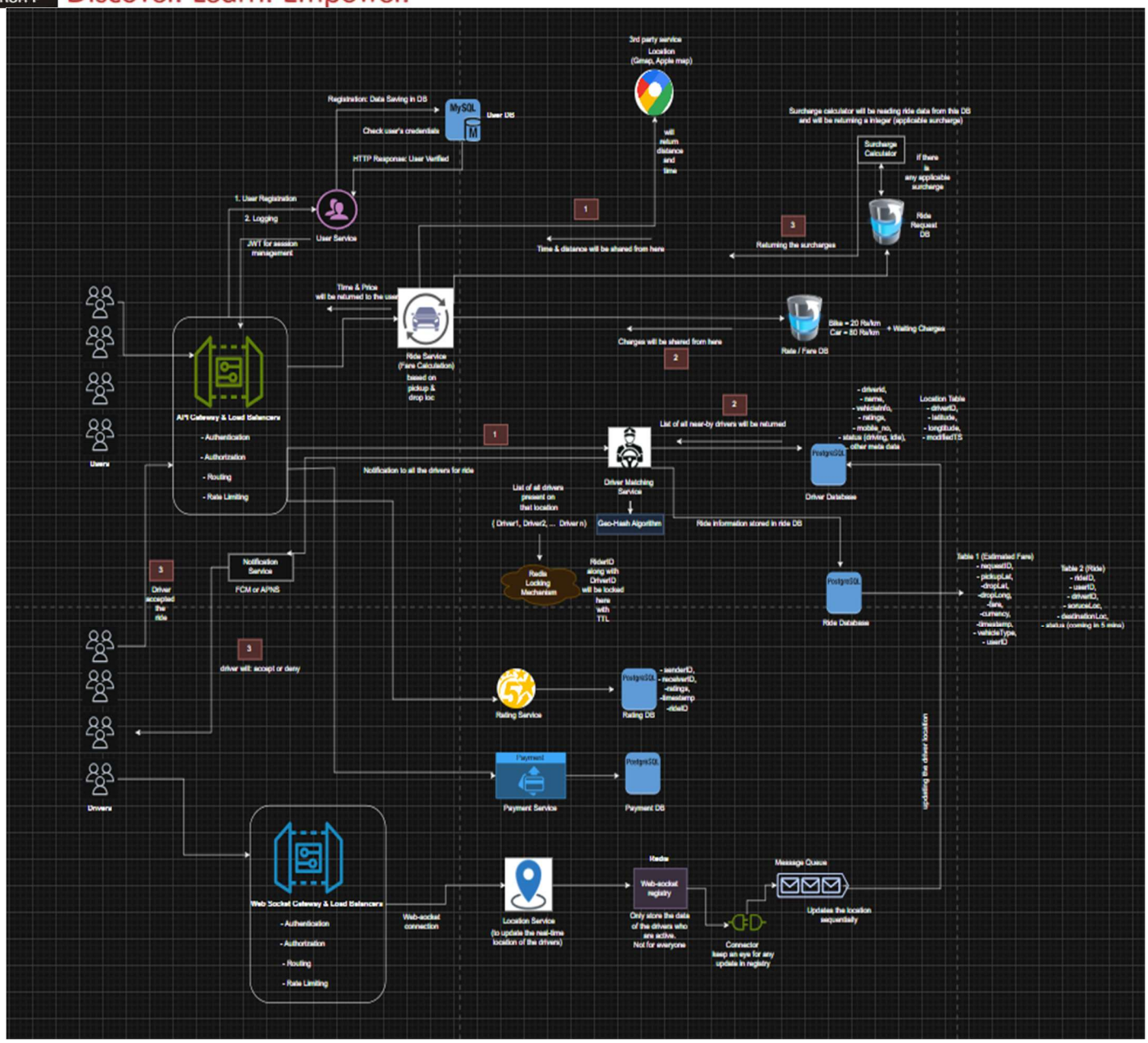
{
"timestamp": "2026-03-22T18:00:00Z",
"end lat": 30.8500,
"end_long": 76.8500,
"total_distance_km": 12.4,
"waiting_time_minutes": 5,
"toll_charges": 50.00
}

Non-Functional Requirements:

- Scalability
  - Target Users: 1.5 Million users and drivers
  - Concurrent ride requests: High during peak hours
  - System should scale horizontally
- Availability (CAP Theorem)
  - System should prefer High Availability and Partition Tolerance.
  - Follow Eventual Consistency for ride status updates.
- Latency
  - Ride matching latency: < 2 seconds
  - Location update latency: near real-time
- Security
  - Secure authentication (JWT-based).
  - Secure payment processing.
  - Data encryption using HTTPS/TLS.
- High Availability
  - Load balancing across servers.
  - Fault tolerance and redundancy mechanisms.

High Level Design: -

The system consists of Client Applications, an API Gateway, and microservices (User Registration, Ride Requests, Driver Matching, Notifications, Ratings, and Payments). Load Balancers manage authentication, authorization, routing, and rate limiting. Payment flows connect to secure services with dedicated databases. Ride-matching and notifications run through asynchronous pipelines for real-time responsiveness. Data is stored in specialized databases for users, drivers, rides, ratings, and payments, ensuring scalability and reliability.



Outcome:

The ride-sharing system was successfully designed with key features like ride booking, driver matching, tracking, and payments, ensuring scalability, low latency, and high availability for real-world usage.